

Device Backend API Documentation

Overview

The device backend system provides a unified interface for executing BDI graphs on heterogeneous hardware: CPUs, GPUs (via OpenCL), and FPGAs (via Verilog synthesis).

Architecture

```
DeviceVTable (Abstract Interface)
├─ CPU Backend (Direct Execution)
├─ GPU Backend (OpenCL)
└─ FPGA Backend (Verilog Generation)
```

Device VTable Interface

```
typedef struct {
    DeviceId id;
    const char* name;
    int (*lower)(const GraphNode* node, void** out_kernel);
    int (*enqueue)(const void* kernel, const HamRegion** regions, size_t num_regions);
    int (*sync)(void);
} DeviceVTable;
```

Methods

lower: Compile BDI node to device-specific kernel

enqueue: Submit kernel for execution

sync: Wait for all pending operations to complete

CPU Backend

Features

- Direct function pointer execution
- SIMD-ready implementations
- Zero-copy memory access
- Synchronous execution

API

```
int cpu_lower(const GraphNode* node, void** out_kernel);
int cpu_enqueue(const void* kernel, const HamRegion** regions, size_t num_regions);
int cpu_sync(void);
```

Supported Operations

OpCode	Implementation	Performance
OP_ADD	Vectorized addition	~10 GFLOPS
OP_MUL	Vectorized multiplication	~10 GFLOPS
OP_RELU	Conditional assignment	~15 GFLOPS
OP_MATMUL	Blocked matrix multiply	~5 GFLOPS

Example

```
#include "kernel/backend/cpu/cpu_backend.h"

// Create node
GraphNode node = {
    .opcode = OP_ADD,
    .type = {.bit_width = 32, .fp = 1}
};

// Lower to CPU kernel
void* kernel = NULL;
if (cpu_lower(&node, &kernel) == 0) {
    // Prepare memory regions
    HamRegion* regions[] = {input_a, input_b, output};

    // Execute
    cpu_enqueue(kernel, regions, 3);
    cpu_sync();

    free(kernel);
}
```

GPU Backend (OpenCL)

Features

- OpenCL 1.2+ compatibility
- Automatic buffer management
- Asynchronous execution
- Multi-device support

API

```
int gpu_init(void);
void gpu_shutdown(void);
int gpu_lower(const GraphNode* node, void** out_kernel);
int gpu_enqueue(const void* kernel, const HamRegion** regions, size_t num_regions);
int gpu_sync(void);
```

Buffer Management

```
GpuBuffer* gpu_buffer_create(size_t size);
void gpu_buffer_free(GpuBuffer* buffer);
int gpu_buffer_write(GpuBuffer* buffer, const void* data, size_t size);
int gpu_buffer_read(GpuBuffer* buffer, void* data, size_t size);
```

OpenCL Kernel Templates

Addition:

```
__kernel void add(__global const float* a,
                  __global const float* b,
                  __global float* c,
                  int n) {
    int i = get_global_id(0);
    if (i < n) c[i] = a[i] + b[i];
}
```

ReLU:

```
__kernel void relu(__global const float* a,
                   __global float* b,
                   int n) {
    int i = get_global_id(0);
    if (i < n) b[i] = (a[i] > 0.0f) ? a[i] : 0.0f;
}
```

Example

```
#include "kernel/backend/gpu/gpu_backend_openc1.h"

// Initialize GPU
if (gpu_init() != 0) {
    fprintf(stderr, "GPU initialization failed\n");
    return -1;
}

// Lower node to GPU kernel
GraphNode node = {.opcode = OP_MUL};
void* kernel = NULL;
gpu_lower(&node, &kernel);

// Execute on GPU
gpu_enqueue(kernel, regions, num_regions);
gpu_sync();

// Cleanup
free(kernel);
gpu_shutdown();
```

FPGA Backend (Verilog)

Features

- Automatic Verilog generation

- Pipelined arithmetic units
- Resource usage estimation
- Latency prediction

API

```
int fpga_lower(const GraphNode* node, void** out_kernel);
int fpga_synthesize_subgraph(const BdiGraph* graph,
                             const Subgraph* subgraph,
                             FpgaKernel** out_kernel);
int fpga_enqueue(const void* kernel, const HamRegion** regions, size_t num_regions);
int fpga_sync(void);
```

Verilog Generation

Addition Module:

```
module add_123 (
    input wire clk,
    input wire rst,
    input wire [31:0] a,
    input wire [31:0] b,
    output reg [31:0] c
);
    always @(posedge clk) begin
        if (rst) c <= 32'b0;
        else c <= a + b;
    end
endmodule
```

Resource Estimation

Operation	LUTs	FFs	DSPs	Latency (cycles)
ADD	100	32	0	1
MUL	300	64	1	3
RELU	50	32	0	1

Example

```
#include "kernel/backend/fpga/fpga_verilog.h"

// Generate Verilog for node
GraphNode node = {.id = 42, .opcode = OP_ADD};
void* kernel = NULL;

if (fpga_lower(&node, &kernel) == 0) {
    FpgaKernel* fpga_kernel = (FpgaKernel*)kernel;

    printf("Generated Verilog (%zu bytes):\n%s\n",
           fpga_kernel->code_len,
           fpga_kernel->verilog_code);

    printf("Latency: %u cycles\n", fpga_kernel->latency_cycles);
    printf("Resources: %u LUTs\n", fpga_kernel->resource_usage);

    fpga_kernel_free(fpga_kernel);
}
```

Multi-Device Execution

Device Selection Strategy

```
DeviceVTable* select_device(const GraphNode* node,
                           DeviceVTable** devices,
                           size_t device_count) {
    // Heuristic: Use GPU for large parallel ops
    if (node->opcode == OP_MATMUL &&
        node->type.vector_len > 1024) {
        return find_device(devices, device_count, DEVICE_ID_GPU);
    }

    // Use FPGA for fixed-function pipelines
    if (node->opcode == OP_SUBGRAPH_BEGIN) {
        return find_device(devices, device_count, DEVICE_ID_FPGA);
    }

    // Default to CPU
    return find_device(devices, device_count, DEVICE_ID_CPU);
}
```

Heterogeneous Execution

```
void execute_heterogeneous(BdiGraph* graph,
                          DeviceVTable** devices,
                          size_t device_count) {
    for (size_t i = 0; i < graph->node_count; i++) {
        GraphNode* node = &graph->nodes[i];

        // Select best device for this node
        DeviceVTable* device = select_device(node, devices, device_count);

        // Lower and execute
        void* kernel = NULL;
        if (device->lower(node, &kernel) == 0) {
            device->enqueue(kernel, NULL, 0);
            free(kernel);
        }
    }

    // Sync all devices
    for (size_t i = 0; i < device_count; i++) {
        devices[i]->sync();
    }
}
```

Performance Tuning

CPU Optimization

- Use SIMD intrinsics for vectorization
- Align memory to cache line boundaries
- Minimize branch mispredictions

GPU Optimization

- Maximize occupancy (threads per SM)
- Coalesce memory accesses
- Use shared memory for frequently accessed data

FPGA Optimization

- Pipeline long combinational paths
- Use block RAM for large buffers
- Balance resource usage vs. throughput

Error Handling

```
int execute_with_fallback(const GraphNode* node,
                          DeviceVTable** devices,
                          size_t device_count) {
    for (size_t i = 0; i < device_count; i++) {
        void* kernel = NULL;

        if (devices[i]->lower(node, &kernel) == 0) {
            if (devices[i]->enqueue(kernel, NULL, 0) == 0) {
                devices[i]->sync();
                free(kernel);
                return 0; // Success
            }
            free(kernel);
        }
    }

    return -1; // All devices failed
}
```

See Also

- [Graph Optimization Guide: Preparing graphs for execution](#)
- [Scheduler Guide: Orchestrating multi-device execution](#)
- [HAM Intelligence Guide: Memory management for devices](#)